Cerca informada

PROP 2021-22 Q2

Mario Martin – Ignasi Gómez Sebastià – Bernat Orellana

Graf

- Modelem els problemes de cerca com a grafs
- Implementació pròpia
  - Expandir per poder tractar el problema abordat

```
public class Node {
    public String name;
    public List<Edge> connections;
}
```

```
public class Edge {
    public Node start;
    public Node end;
    public double weight;
}
```

```
public class Graph {
    List<Node> nodes;
}
```

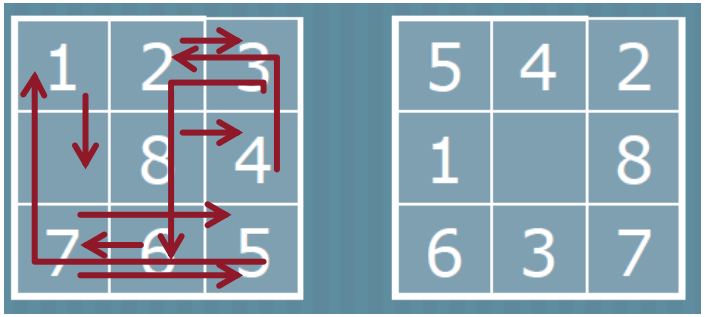
- Llibreria externa
  - http://jung.sourceforge.net/doc/index.html

Algorismes de cerca informats

- Els algorismes de cerca informats utilitzen coneixement del problema per prioritzar els nodes més prometedors.
- Aquesta informació s'anomena **Heurística**. És una funció que val zero si estem a la solució i és creixent com major sigui el cost estimat pendent per arribar a la solució.

Algorismes de cerca informats

- Tasques:
  - Programar heurístics per el 9-puzle a la classe *NinePuzzle*
    - Funció int **getHeuristica_1()** : indica el nombre de caselles mal colocades respecte la solució (sense comptar l'espai)
    - Funció int **getHeuristica_2()**:indica la distància de Manhatan de cada número de de l'objectiu fins a la solució.



H1=8 (tots mal colocats)
H2=15 (suma long. fletxes)

Algorismes de cerca informats

- Els algorismes de cerca requereixen treballar sobre una llista de nodes ordenada segons l'heurístic.

```
def hillClimbing(graph, start, heuristic):
    LNT = []
    LNO = [start]
    solucio = False
    while len(LNO) > 0 and not solucio:
        vertex = stack.pop()
        if vertex not in LNT:
            if not  isSolucio(vertex):
                LNT.add(vertex)
                LF = graph[vertex]
                LF = LF.sort(heuristic)[0]
                LNO.expand(LF - LNT)
            else:
                solucio = True
    tractar_solucio(vertex)
```

```
def bestFirst(graph, start, heuristic):
    LNT = []
    LNO = [start]
    solucio = False
    while len(LNO) > 0 and not solucio:
        vertex = stack.pop()
        if vertex not in LNT:
            if not  isSolucio(vertex):
                LNT.add(vertex)
                LF = graph[vertex]
                LNO.expand(LF - LNT)
                LNO = LNO.sort(heuristic)
            else:
                solucio = True
    tractar_solucio(vertex)
```

Algorismes de cerca informats

- Si matenim les nostres llistes ordenades per heurístic, podem treballar fent inserció ordenada que és molt més eficient que ordenar tot cada cop.
- Podem usar **Collections.binarySearch** i un ***Comparator*** per trobar el punt d'inserció:

binarySearch

```
public static <T> int binarySearch(List<? extends T> list,
                   T key,
                   Comparator<? super T> c)
```

Searches the specified list for the specified object using the binary search algorithm. The list must be sorted into ascending order according to the specified comparator (as by the `sort(List, Comparator)` method), prior to making this call. If it is not sorted, the results are undefined. If the list contains multiple elements equal to the specified object, there is no guarantee which one will be found.

This method runs in log(n) time for a "random access" list (which provides near-constant-time positional access). If the specified list does not implement the `RandomAccess` interface and is large, this method will do an iterator-based binary search that performs O(n) link traversals and O(log n) element comparisons.

Parameters:

`list` - the list to be searched.

`key` - the key to be searched for.

`c` - the comparator by which the list is ordered. A `null` value indicates that the elements' natural ordering should be used.

Returns:

the index of the search key, if it is contained in the list; otherwise, `(-(insertion point) - 1)`. The *insertion point* is defined as the point at which the key would be inserted into the list: the index of the first element greater than the key, or `list.size()` if all elements in the list are less than the specified key. Note that this guarantees that the return value will be >= 0 if and only if the key is found.

Throws:

`ClassCastException` - if the list contains elements that are not *mutually comparable* using the specified comparator, or the search key is not mutually comparable with the elements of the list using this comparator.

Algorismes de c

- Exemple d'ús de ***Comparator*** i cerca binària.
- El vostre ***Comparator*** pot cridar **getHeuristica_1() o getHeuristica_2().**

```
17  public class DemoBinarySearch {
18
19      // Classe continguda a les llistes
20      private static class Client {
            private int id;
            private String url;
23          // Constructor
24          public Client(int id, String url) {
25              this.id = id;
26              this.url = url;
27          }
28          public Integer getId() {
                return Integer.valueOf(id);
30          }
31      }
32
33      public static void main(String[] args) {
34
35          // Creem una llista de clients
            List<Client> llista = new ArrayList<Client>();
37          llista.add(new Client(10, "Paco Pil"));
38          llista.add(new Client(20, "Joao Peres"));
39          llista.add(new Client(30, "Kenji Sakamoto"));
40          llista.add(new Client(40, "Do Remi"));
41
42          // Comparador per Id
            Comparator<Client> c_per_id = new Comparator<Client>() {
                public int compare(Client u1, Client u2) {
45                  return u1.getId().compareTo(u2.getId());
46              }
47          };
48
49          // Si el trobem, retorna la posició (1 en aquest cas)
50          int index = Collections.binarySearch(llista,
51                  new Client(10, null), c_per_id);
52          System.out.println("Found at index  " + index);
53
54          // Si no es troba l'element, retorna un valor negatiu:
55          //  (-(insertion point) - 1) =
56          //   -(3)-1 = -4
57          index = Collections.binarySearch(llista,
58                  new Client(35, "Nou client"), c_per_id);
59          if(index<0) {
60              int puntInsercio = -(index+1);
61              System.out.println("Punt d'inserció:" +puntInsercio);
62          }
63      }
64  }
```

Algorismes de cerca informats

- Implementar un algoritme de Hill Climbing
  - Troba una solució?
  - Quina mida té la solució?
- Implementar un algorisme de BeamSearch
  - Proveu amb diferents mides de Beam. Té incidència amb la mida de la solució ?
- Implementeu A*